

Assignment 18.4 Ai Assisted Coding

Htno:2303A510C1

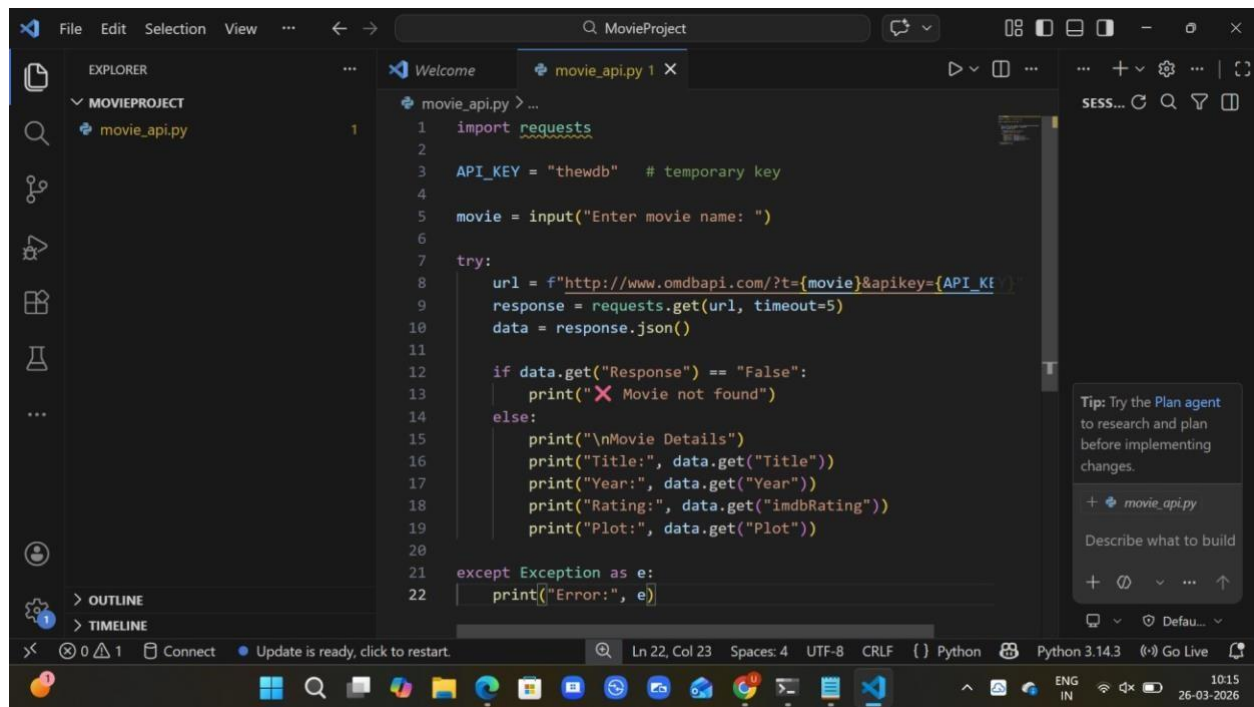
Btno:06

Task 1: Movie Database API Integration

Prompt:Generate a Python script to fetch movie details using OMDB API.

The program should accept movie title as input and display title, year, rating, and plot. Include proper error handling for invalid movie names, API errors, and network issues.

Code:



```
1 import requests
2
3 API_KEY = "thewdb" # temporary key
4
5 movie = input("Enter movie name: ")
6
7 try:
8     url = f"http://www.omdbapi.com/?t={movie}&apikey={API_KEY}"
9     response = requests.get(url, timeout=5)
10    data = response.json()
11
12    if data.get("Response") == "False":
13        print("❌ Movie not found")
14    else:
15        print("\nMovie Details")
16        print("Title:", data.get("Title"))
17        print("Year:", data.get("Year"))
18        print("Rating:", data.get("imdbRating"))
19        print("Plot:", data.get("Plot"))
20
21 except Exception as e:
22     print(f"Error: {e}")
```

The screenshot shows a Visual Studio Code editor window with a file named 'movie_api.py' open. The code is a Python script that uses the 'requests' library to fetch movie details from the OMDB API. It prompts the user for a movie name, constructs a URL with the movie title and a temporary API key, and then makes a GET request. The response is parsed as JSON, and the script checks if the movie was found. If not, it prints an error message. If found, it prints the movie's title, year, rating, and plot. The script also includes a try-except block to handle any exceptions that might occur during the API call. The status bar at the bottom indicates the current cursor position (Ln 22, Col 23), the file encoding (UTF-8), the line ending (CRLF), and the Python version (Python 3.14.3).

Output:

```
PS C:\Users\uppus\Downloads\MovieProject> python movie_api.py
Enter movie name: Avatar

Movie Details
Title: Avatar
Year: 2009
Rating: 7.9
Plot: A paraplegic Marine dispatched to the moon Pandora on a unique mission becomes torn between following his orders and protecting the world he feels is his home.

PS C:\Users\uppus\Downloads\MovieProject>
```

Explanation:

-->The program uses requests library to connect to OMDB API.

-->It takes movie name as user input.

-->The API response is converted into JSON format.

-->Required details like title, year, rating, and plot are displayed.

-->Error handling is implemented for invalid movie names and network issues.

Task 2: Public Transport Schedule API Integration

Prompt: Generate a Python script to fetch public transport arrival timings using an API.

The program should accept a stop ID as input and display the next 3 arrivals.

Include error handling for invalid stop ID, server issues, and timeout errors.

Also implement retry logic for failed API calls.

Code:

This screenshot shows the first part of the `transport_api.py` script in a VS Code editor. The Explorer pane on the left shows the project structure with `movie_api.py` and `transport_api.py`. The code in the editor includes imports, a user input for a stop ID, a URL construction, and a loop for fetching data from the TfL API. A right-hand sidebar contains a tip about the Plan agent and a search bar.

```
1 > import requests ...
3
4 stop_id = input("Enter Stop ID (example: 49008660N): ")
5
6 url = f"https://api.tfl.gov.uk/StopPoint/{stop_id}/Arrivals"
7
8 for attempt in range(2):
9     try:
10         print("Fetching data...")
11
12         response = requests.get(url, timeout=10) # increased timeout
13
14         if response.status_code != 200:
15             print("❌ Invalid Stop ID or Server Error")
16             break
17
18         data = response.json()
19
20         if not data:
21             print("No arrivals found")
22             break
23
24         print("\n📄 Next 3 Arrivals")
```

This screenshot shows the second part of the `transport_api.py` script, focusing on exception handling. It includes try-except blocks for `requests.exceptions.Timeout` and `requests.exceptions.ConnectionError`, as well as a general `Exception` handler. The right-hand sidebar is identical to the one in the first screenshot.

```
30         break
31
32     except requests.exceptions.Timeout:
33         print("⌚ Timeout... Retrying")
34         time.sleep(2)
35
36     except requests.exceptions.ConnectionError:
37         print("🌐 Network error. Check internet")
38         break
39
40     except Exception as e:
41         print("Error:", e)
42         break
```

Output:

```
Enter Stop ID (example: 490008660N): 490008660N
Fetching data...
```

```
🚗 Next 3 Arrivals
```

```
🚗 Next 3 Arrivals
```

```
-----
```

```
214 - 2026-03-26T05:02:24Z
```

```
88 - 2026-03-26T05:16:28Z
```

Explanation:

-->The program uses the requests library to connect to a transport API and fetch arrival data.

-->It takes stop ID as user input to retrieve specific station details.

-->The API response is converted into JSON format and processed.

-->It displays the next 3 arrival timings in a readable format.

Task 3: Stock Market Data API Integration

Prompt: Generate a Python script to fetch stock market data using a financial API.

The program should accept a stock symbol as input and display current price, day high, and day low. Include error handling for invalid symbols, API rate limits, and missing data in the API response.

Code:

The screenshot shows the Visual Studio Code editor with a project named 'MovieProject'. The Explorer sidebar on the left shows three files: 'movie_api.py', 'stock_api.py', and 'transport_api.py'. The 'stock_api.py' file is open in the editor, displaying the following Python code:

```
1 import requests
2
3 # Put your API key here
4 API_KEY = "demo" # you can use "demo" for testing
5
6 symbol = input("Enter stock symbol (example: IBM): ")
7
8 try:
9     url = f"https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol={symbol}&apikey={API_KEY}"
10
11     response = requests.get(url, timeout=5)
12     data = response.json()
13
14     stock = data.get("Global Quote", {})
15
16     # Invalid symbol
17     if not stock:
18         print("Invalid stock symbol or no data found")
19
20     else:
21         print("\n Stock Details")
22         print("-----")
23         print("Price      :", stock.get("05. price", "N/A"))
24         print("Day High   :", stock.get("03. high", "N/A"))
25         print("Day Low    :", stock.get("04. low", "N/A"))
26
27 # Timeout
28 except requests.exceptions.Timeout:
29     print("Request timed out")
30
31 # Network error
32 except requests.exceptions.ConnectionError:
33     print("Network error")
34
35 # Other errors
36 except Exception as e:
37     print("Error:", e)
```

The status bar at the bottom indicates the file is at line 37, column 23, using UTF-8 encoding with CRLF line endings. The Python version is 3.14.3. A tip on the right suggests using the Plan agent for research and planning.

This screenshot shows the same Visual Studio Code editor with the 'stock_api.py' file open. The code is identical to the previous screenshot, but the view is scrolled down to show the exception handling logic. The code includes try-except blocks for 'requests.exceptions.Timeout', 'requests.exceptions.ConnectionError', and a general 'Exception' handler. The status bar and right-hand tip remain the same.

Output:

```
PS C:\Users\uppus\Downloads\MovieProject> python stock_api.py
Enter stock symbol (example: IBM): IBM

📈 Stock Details
-----
Price      : 241.3900
Day High   : 246.1900
Day Low    : 238.0000
PS C:\Users\uppus\Downloads\MovieProject> 
```

Explaantion:

- >The program uses the requests library to fetch stock data from a financial API.
- >It takes stock symbol as input from the user (e.g., IBM).
- >The API response is converted into JSON format and processed safely.
- >It displays important details like current price, day high, and day low.
- >Error handling is implemented for invalid symbols, network issues, and missing JSON fields.

Task 4: Geolocation API (IP Address Lookup)

Prompt: Generate a Python script to fetch geolocation details using an IP address.

The program should accept an IP address as input and display country, city, and ISP.

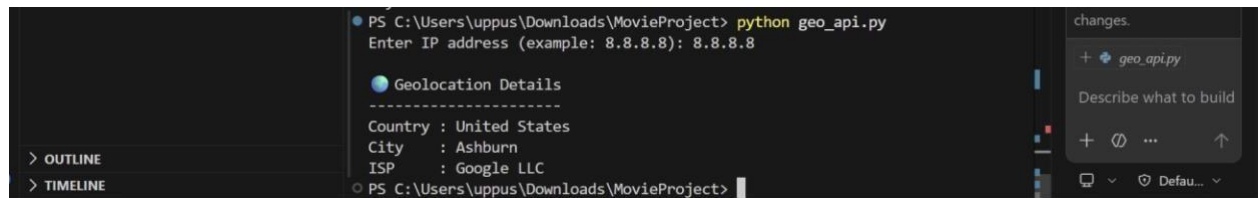
Include validation for IP format and error handling for invalid input, API failure, and JSON decoding errors.

Code:


```
1 import requests
2 import re
3
4 # Take IP input
5 ip = input("Enter IP address (example: 8.8.8.8): ")
6
7 # Validate IP format
8 pattern = r"^\d{1,3}(\.\d{1,3}){3}$"
9
10 if not re.match(pattern, ip):
11     print("❌ Invalid IP format")
12 else:
13     try:
14         url = f"http://ip-api.com/json/{ip}"
15         response = requests.get(url, timeout=5)
16
17         # Convert to JSON
18         data = response.json()
19
20         # Check API status
21         if data.get("status") != "success":
22             print("❌ Failed to fetch location")
23         else:
```

```
23         print("\n📍 Geolocation Details")
24         print("-----")
25         print("Country :", data.get("country", "N/A"))
26         print("City   :", data.get("city", "N/A"))
27         print("ISP    :", data.get("isp", "N/A"))
28
29     except requests.exceptions.Timeout:
30         print("⌚ Request timed out")
31
32     except requests.exceptions.ConnectionError:
33         print("🌐 Network error")
34
35     except ValueError:
36         print("⚠️ JSON decoding error")
37
38     except Exception as e:
39         print(f"Error: {e}")
40
```

Output:



The screenshot shows a VS Code interface. The terminal window on the left displays the command `python geo_api.py` and the prompt `Enter IP address (example: 8.8.8.8): 8.8.8.8`. Below the prompt, the output shows 'Geolocation Details' followed by a dashed line and the following information: Country : United States, City : Ashburn, and ISP : Google LLC. The Explorer on the right shows a file named `geo_api.py` under a folder named 'changes'.

Explanation:

- >The program accepts an IP address as user input.
- >It validates the IP format using regular expressions (regex).
- >It uses the requests library to connect to a geolocation API.
- >The API response is converted into JSON format and parsed.
- >Error handling is implemented for invalid IP, network issues, API failure, and JSON errors.

Task 5: Real-Time Application – Payment Gateway Simulation API

Prompt: Generate a Python script to simulate payment processing using a sandbox API.

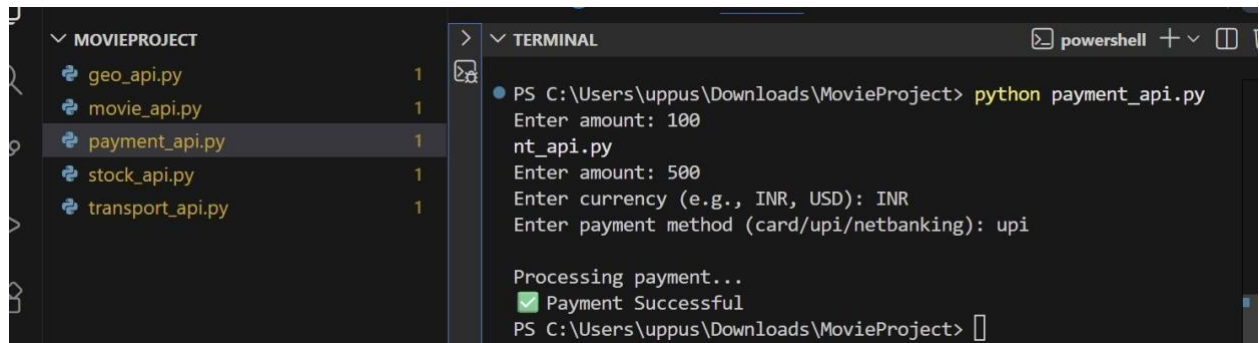
The program should send a POST request with amount, currency, and payment method. Include input validation and handle errors such as invalid payment details, HTTP status errors (400, 401, 500), and network issues. Also log failed transactions into a file.

Code:


```
1 import requests
2
3 # User input
4 amount = input("Enter amount: ")
5 currency = input("Enter currency (e.g., INR, USD): ")
6 method = input("Enter payment method (card/upi/netbanking): ")
7
8 # Basic validation
9 if not amount.isdigit():
10     print("❌ Invalid amount")
11 else:
12     try:
13         url = "https://httpbin.org/post"
14
15         payload = {
16             "amount": amount,
17             "currency": currency,
18             "method": method
19         }
20
21         print("\nProcessing payment...")
22
23         response = requests.post(url, json=payload, timeout=5)
```

```
34 elif response.status_code == 500:
35     print("❌ Server Error")
36 else:
37     print("❌ Payment Failed with status:", response.status_code)
38
39 # Timeout
40 except requests.exceptions.Timeout:
41     print("⌚ Request timed out")
42
43 # Network error
44 except requests.exceptions.ConnectionError:
45     print("🌐 Network error")
46
47 # ! Other errors + logging
48 except Exception as e:
49     print("Error:", e)
50
51 # Log error to file
52 with open("error_log.txt", "a") as f:
53     f.write(str(e) + "\n")
```

Output:



```
MOVIEPROJECT
├── geo_api.py
├── movie_api.py
├── payment_api.py
├── stock_api.py
└── transport_api.py
```

```
PS C:\Users\uppus\Downloads\MovieProject> python payment_api.py
Enter amount: 100
nt_api.py
Enter amount: 500
Enter currency (e.g., INR, USD): INR
Enter payment method (card/upi/netbanking): upi

Processing payment...
✔ Payment Successful
PS C:\Users\uppus\Downloads\MovieProject>
```

Explanation:

- >The program collects payment details (amount, currency, method) from the user.
- >It sends a POST request to a sandbox API to simulate payment processing.
- >Input validation is performed to ensure the amount is valid before processing.
- >It handles errors such as HTTP status errors (400, 401, 500), timeout, and network issues.
- >Failed transactions are logged into a file for future reference and debugging.